AI Engineering and Claude Architect · Part IV, Security

AI Security, Lesson 5

Jailbreaks, Guardrails and Responsible AI

A screen that catches 95% of attacks and wrongly flags 1% of messages sounds excellent. On real traffic, fewer than one flag in ten is a real attack. Guardrails are a base-rate problem.

Module	AI09 of 13
Lesson	5 of 5
Prerequisites	AI09 L1–L2, AI02 L4, AI08 L7
Exam relevance	Mostly out of scope. Supports TS 1.4, 4.3, 4.4, 5.2, 5.5
Reading time	~35 minutes

Contents

1. Jailbreaks, Guardrails and Responsible AI
2. Module Revision

Lesson 5 of 5 · AI Security

Jailbreaks, Guardrails and Responsible AI

Key idea

A guardrail is a classifier, so judge it by its errors at your traffic's real attack rate. Layer input and output checks, keep the expensive checks for risky paths, and prefer checks against facts over checks against feelings. Then decide who's accountable when it fails, since it will.

> *Before: threat modelling, RLHF and Constitutional AI, testing. Teaches: jailbreak classes, classifier defenses and their published results, the base-rate problem, output checks, abuse limits, and responsible AI practice. Used by: AI10, AI11.*

How this lesson handles attacks. It names classes of attack, as published by defenders, and gives no prompts that work.

The hook: the 95% screen

A team buys a content screen that catches 95% of attacks with a 1% false-alarm rate. They ship it. A week later, support is flooded by people wrongly blocked, and the security analyst has stopped reading alerts.

Nothing was misreported. Here are those rates on a million messages. **All rates are illustrative inputs I chose,** and this is real output from `GuardrailMath.java`:

```
A million messages, illustrative rates:
rare attacks, good screen  attacks  1000, missed   50, innocent blocked  9990, flags that are real  8.7%
rare attacks, strict       attacks  1000, missed   10, innocent blocked 49950, flags that are real  1.9%
rare attacks, lenient      attacks  1000, missed  200, innocent blocked   999, flags that are real 44.5%
common attacks             attacks 50000, missed 2500, innocent blocked  9500, flags that are real 83.3%
```

Look at the first row. With attacks at one in a thousand, **fewer than one flag in eleven is a real attack, on a screen with 95% recall.** That's the base-rate effect, the same arithmetic that makes screening tests for rare diseases hard. Going stricter, in row two, makes it worse: only 1.9% of flags are real.

So a "95% accurate" guardrail can still be a bad product decision, and the right response depends on what a flag **does**. A hard block for 9,990 innocent people is bad. Sending those messages to a closer look may be fine.

Intuition: airport security

An airport has several layers: a check-in desk, a scanner, a closer look for anything flagged, and a locked cockpit door. The scanner flags many harmless bags, so a flag leads to a second look, and doesn't lead to arrest.

And the locked door is the control that assumes everything else failed. L2 and L3 built the locked door. This lesson is about the scanners, which remain useful for lowering how often anyone reaches it.

Mechanism: jailbreaks

Injection and jailbreak differ

	Jailbreak	Prompt injection
Goal	Get the model to produce what it's trained to refuse	Get the model to follow someone else's instructions
Usually comes from	The user, directly	Content the agent reads
Harm is	The output itself	The actions taken, or the data stolen
Main defense	Model training and classifiers	System design (L2, L3)
Whose responsibility	Largely the model's provider, plus your screens	Largely yours

They overlap, and Anthropic's guide treats them together, but the distinction changes what you build.

Why they exist

AI02 L4 taught RLHF and Constitutional AI. Refusal behaviour is learned, on top of a model that's capable of much more, so it can be argued with. There's no separate rule engine underneath. A model is also trained to be helpful, follow instructions and play roles, and jailbreaks exploit those useful traits pulling against the trained refusal.

Classes that defenders have published

Class	Idea, without a recipe	Source
Encoding and ciphers	Hide the request in a form that safety training saw less often	Constitutional Classifiers
Role-play	Frame it as fiction or a character	Constitutional Classifiers
Keyword substitution	Swap the sensitive words	Constitutional Classifiers
Prompt injection	Instructions disguised as authoritative	Constitutional Classifiers
Many-shot	Long context filled with faked dialogues in which the assistant complies	Many-shot jailbreaking

Many-shot is a good lesson about capability trade-offs. Anthropic's research ran up to 256 faked dialogues in one prompt, on Claude 2.0, and success followed a power law with the number of shots. Longer context windows, a feature people want, opened the attack. Its results on defenses:

Defense	Result	What it shows
Limit context length	Fully prevents it, at a cost in utility	Blunt, and it works
Fine-tuning to refuse	"merely delayed the jailbreak"	Training raises the bar, without removing it
Classify and modify the prompt	"dropping the attack success rate from 61% to 2%"	Layered systems work

Those are 2024 figures for that model and setup. Don't apply them to current models.

Constitutional Classifiers: a published defense in detail

Anthropic's work here is the best public example of a guardrail measured in depth, so it's worth reading its numbers carefully.

How it works. A "constitution" lists allowed and disallowed content classes. From that, a model generates synthetic training data, including translated and jailbreak-styled variants. Classifiers then screen both inputs and outputs.

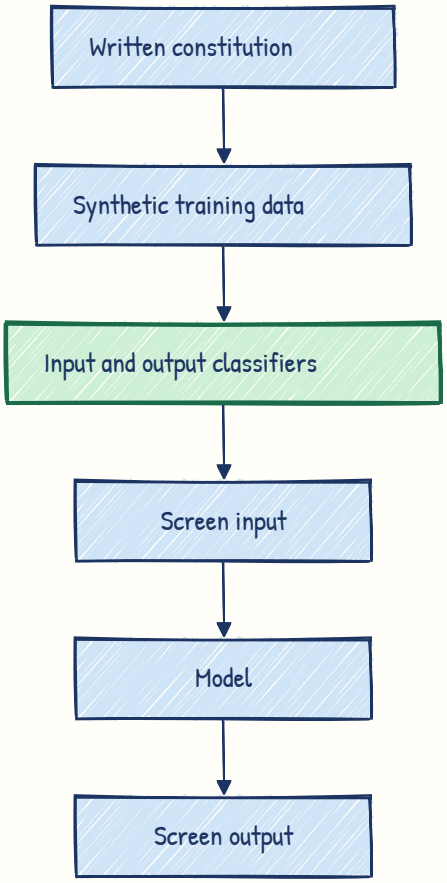


Figure 1. Classifiers are trained from a written policy, and screen both directions. The model in the middle stays unchanged.

Results, from Anthropic's article of February 2025:

Test	Result
Human red team on a prototype	183 participants, over 3,000 hours in two months, for up to $15,000
Outcome	"none of the participants were able to coerce the model to answer all ten forbidden queries with a single jailbreak." The prototype over-refused and was expensive
Automated, 10,000 synthetic jailbreaks	86% success without classifiers, **4.4% with** (Claude 3.5 Sonnet, October 2024)
Cost: refusals	Up 0.38% on 5,000 conversations, which wasn't statistically significant
Cost: compute	Up 23.7%
Public demo, 3–10 February 2025	339 jailbreakers, over 300,000 chats, about 3,700 hours
Demo outcome	None passed all eight questions in the first five days. Then four cleared them, with one universal jailbreak, and one borderline

Three things to take from this:

→ **It's excellent, and it wasn't perfect.** The strong result fell to determined people within a week. Anthropic says it may not stop every universal jailbreak, and recommends complementary defenses.

→ **The trade-off was measured.** Refusals and compute are in the results, which most vendor claims leave out. Ask for these from any guardrail.

→ **Scope.** The target was a narrow set of dangerous content, chosen by the researchers. Your needs are different, so these rates aren't yours.

Guardrails you build

The guide on mitigating jailbreaks gives a direct pattern: a **harmlessness screen**, using a small fast model with structured output returning a boolean, placed before the main model. It recommends a Haiku model (Haiku 5.5 in its example), and it treats `stop_reason: "refusal"` from the screen as a harmful verdict.

That second part is a neat trap. A screen that refuses isn't returning `{"harmful": false}`. If your code reads a missing field as safe, then the worst inputs pass. **A screen that doesn't return a verdict has to fail closed.**

Where checks go:

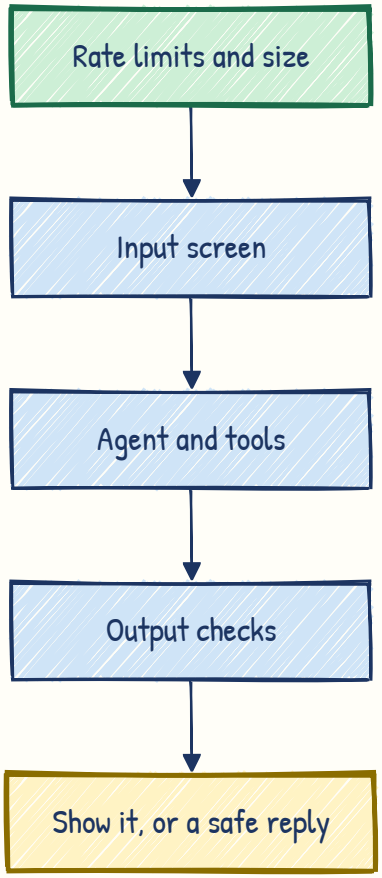


Figure 2. Cheap checks run first. Each stage can stop the request, and only risky paths pay for the expensive ones.

Type of check	Examples	Good at	Weak at
Rules	Length, rate, format, allowlist	Cheap, exact	Meaning
Facts	Figures match tool results, citations resolve	Exact, and explainable	Tone and judgment
Classifier	Topic, harm, injection	Meaning	Errors at base rate
Model judge	"Does this follow the policy?"	Nuance	Cost, and it's fallible itself
Human	Sampled or escalated	Judgment	Fatigue, and scale

Prefer checks against facts

Here's a fact check, comparing figures in an answer with tool results. Real output:

```
You spent 41.30 on groceries.                    unsupported: []
You spent 43.10 on groceries.                    unsupported: [43.10]
You spent 1.30 on groceries.                     unsupported: [1.30]
You spent about forty dollars.                   unsupported: []
You spent 41.30, and you should cancel your card.  unsupported: []
```

The first three are correct, and it needs no model. My earlier version used a substring match, and passed 1.30 because it's inside 41.30, which I corrected before publishing. The last two rows show the limits. It doesn't read words, so "forty dollars" passes, and so does advice to cancel a card. **A check only covers what it checks,** so say which risks remain.

The same logic covers citations (does the document exist, and does this quote appear in it?) and structured output (TS 4.3 removes syntax errors, not meaning errors).

Streaming complicates output checks

Once text is on a user's screen, you can't take it back. Options:

Approach	Safety	Experience
Buffer the whole reply, then check	Best	Loses streaming
Buffer by sentence, checking each	Good, and doesn't see the whole context	Slight delay
Stream, and retract on failure	The user has seen it	Odd, but sometimes acceptable
Stream only low-risk flows	Decide by route	Needs routing

This table is an engineering recommendation. I haven't measured it.

Two layers multiply only if they're independent

```
Two screens, each missing 5%:
  missed by both, if independent: 0.25%
  missed by both, if they always fail together: 5.00%
```

Real layers are somewhere between those. An attack crafted to confuse one language model often confuses a similar one, so a second screen of the same type adds less than the arithmetic suggests. Different kinds of check, such as a rule plus a classifier, are more likely to fail differently. Measure their overlap, rather than assuming it.

Limits, abuse and cost

OWASP's LLM10 is Unbounded Consumption. Every token costs you money, so it's a security issue as well:

Control	Stops
Per-user rate and spend limits	One account running up the bill
Input size limits	Very long prompts, including the many-shot class
Step and token limits per run (AI08 L3)	Runaway loops
Tenant budgets	One tenant affecting others
Handling repeat offenders	Anthropic's guide recommends responding to repeated attempts: warn, throttle, or ban

Responsible AI, as engineering

This phrase can drift into vague statements. Here's a version with work in it. It's my synthesis, as an engineering recommendation, and isn't a legal or regulatory standard.

Principle	What you actually build
People know it's AI	Disclosure in the interface, and don't let the bot deny being one
People can reach a human	A real escalation route (TS 5.2), honoured when asked
People can challenge a decision	Reasons recorded, and a way to appeal
Consequential decisions have human oversight	Credit, hiring, medical and legal outputs are advice, with a named accountable person
It works for everyone	Evaluate by segment (TS 5.5). An 83% average can hide a group at 40% (AI08 L7)
It's honest about what it knows	Citations, stated uncertainty, and tested abstention (AI05 L6)
Someone owns it	A named owner, an incident process, and a kill switch

Misinformation is OWASP's LLM09, and it's the common failure for honest users. No attacker is needed. AI05 L6 measures faithfulness, and this lesson's fact checks help.

Fairness needs measuring on your own task. Group language, names and dialects in your evaluation, and compare error rates. No general statement about a model's fairness tells you about your task.

Regulation exists and it changes. I haven't read specific regulations for this course, so I won't state their terms. Ask your compliance team which apply, and treat this table as a place to begin.

Over-refusal is a failure too

A guardrail that blocks legitimate work is a defect. A medical information service that refuses to discuss dosage, or a security team's agent that refuses to explain a CVE, both fail their users, and those users may go somewhere worse. Measure:

Metric	Why
False-positive rate, on real samples	Rate of harm to honest users
Rate by segment	Filters may flag some dialects more than others
Appeals upheld	Direct evidence of mistakes
Attacks missed, from red teaming	The other side

Report both error rates, always. A guardrail reported with only one is being sold to you.

Why the obvious fixes fall short

Obvious idea	Why it falls short
One great screen	Base rates, as in the hook, and attackers adapt
Make the screen stricter	1.9% of flags real, in row two
A second identical screen	Failures correlate
Rely on the provider's safety training	It targets general harms, not your policy
Put policy in the system prompt	Useful, and it's advice. It can leak too
Block everything flagged	Innocent users pay for it
Add a responsible AI statement	A statement isn't an escalation path

How Anthropic solved it

Mechanism	What's published	Status
Constitutional AI	Training from written principles (AI02 L4)	Research and practice
Constitutional Classifiers	Input and output screens, with measured costs	Research with results above
`refusal` stop reason	The API signals when the model refused	Documented behaviour
Screening pattern	Small model plus structured output	Published guidance
Additional classifiers	Extra classifiers on computer and browser use	Stated in the guide
Continuous monitoring	Recommended, with chained safeguards	Published guidance

Anthropic's statement on the limits is that "there are still no fully robust deep-learning models in production." That's the same conclusion as this module's: layer things, and contain what gets through.

How the field approaches it. OWASP's LLM05, 07, 09 and 10 cover output handling, prompt leakage, misinformation and consumption. Many guardrail products exist, and I haven't evaluated any, so I can't recommend one. Ask each for both error rates on your own data.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: the model's own training, and a system prompt.

Improved: an input screen that blocks. New problem: at low attack rates, most blocked people are innocent.

Production: route by risk, with most flags getting a closer look rather than a block.

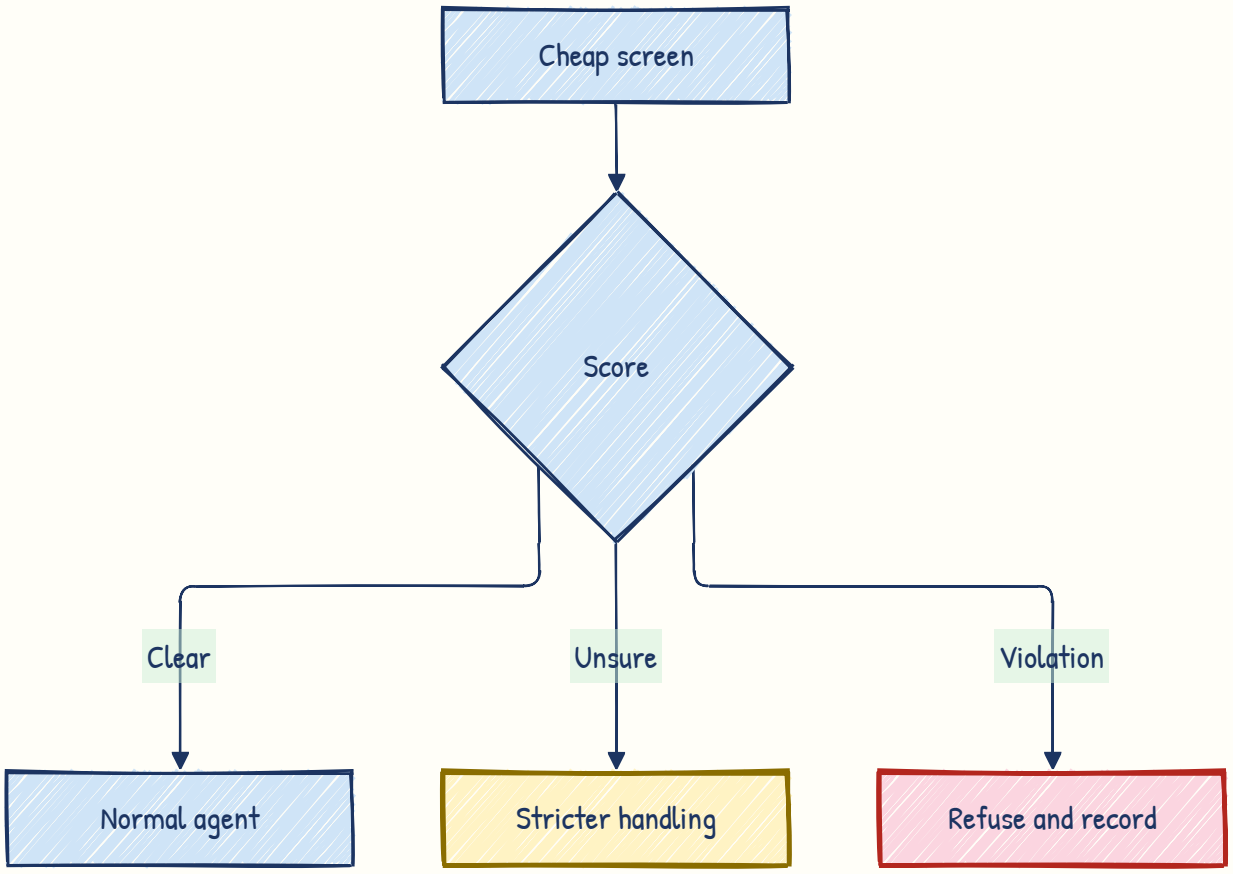


Figure 3. A flag sends a request to a closer look. Only clear violations are blocked, so false alarms cost less.

"Stricter handling" might mean fewer tools, read-only mode, a larger model as judge, or a human.

Component	Why it exists	How it communicates
Rate and spend limits	Cheap and exact	Gateway (AI10)
Input screen	Reduces load on later stages	Small model, or a classifier service
Risk router	Turns scores into handling	Code, with thresholds
Fact checks	Cover figures and citations	Code, using tool results
Output classifier	Covers meaning	Small model
Human queue	For what's unclear	Asynchronous (TS 5.5)
Metrics	Both error rates, by segment	Dashboard (AI10)
Kill switch	Turn off a tool or feature fast	Feature flag

Failures:

Failure	How you'd notice	What to do
Screen service is down	Errors, latency	Fail closed for high risk. Maybe degrade for low risk, and decide in advance
Screen returns no verdict	Parse errors, or a refusal	Treat as flagged
False alarms rise after a release	Flag rate, appeals	Roll back, and sample
New attack class	Red team, or an incident	Add to the test set, and rely on containment meanwhile
Added latency hurts	p95 by stage	Run checks in parallel, and skip on low-risk routes
Guardrail cost grows	Cost per request	Cheap checks first. Sample the expensive ones
Output harm after streaming	User reports	Buffer risky routes

Java

The figure check, with the decision kept visible:

```
Set<String> evidence = new HashSet<>();
toolResults.forEach(result -> evidence.addAll(figures(result)));
return figures(answer).stream().filter(f -> !evidence.contains(f)).toList();
```

It compares whole numbers extracted from each side, which is the fix for the substring bug. `AgentService` already handles the refusal case, returning `Status.REFUSED` rather than treating it as an answer:

```
case "refusal" -> {
    return new Result(Status.REFUSED, turn.text(), step, tokens);
```

The screen itself isn't implemented here, since an honest test needs a real model. Anthropic's guide has the pattern. In AI08's design it would be another `ModelPort` call with a tool schema, and a missing verdict should count as flagged.

Trade-offs

Decision	Benefit	Cost
Input screen	Stops attacks early	Latency, cost, and false alarms
Output check	Sees what would really be shown	Hurts streaming
Hard block	Simple and strong	Harms innocent users
Route by risk	Keeps false alarms cheap	More paths to test
Strict thresholds	Fewer misses	Far fewer of the flags are real
Own classifier	Tuned to your data	Training data and upkeep

When not to add a screen: an internal tool for trusted staff, with no harmful capabilities, may need rate limits and logging only. Don't pay for a guardrail you can't justify with a threat from L1.

Common mistakes

Quoting recall without the false-alarm rate or the base rate. "Catches 95%" says little until you've done the arithmetic for your traffic.

Common mistakes

Treating a missing verdict as safe. A refusal or a parse failure from a screen is exactly when you should be most careful.

Common mistakes

Publishing a responsible AI principle with no ticket behind it. If "a human can review this" isn't a working queue with staff, it's untrue.

What to say in an interview

"A guardrail is a classifier, so I ask for both error rates and do the base-rate arithmetic. At one attack in a thousand, a 95% and 1% screen has about 9% real flags, so a flag should trigger a closer look and not a block. I prefer checks against facts where I can, fail closed on missing verdicts, and measure by segment. And since jailbreaks won't reach zero, tools and data access are contained separately."

Certification lens

How this lesson maps to CCAR-F

Mostly out of scope. The nearby tested ideas are: structured output removes syntax errors and not semantic ones (TS 4.3), validation and retry (TS 4.4), escalation triggers (TS 5.2), human review and accuracy by segment (TS 5.5), and programmatic enforcement (TS 1.4).

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Structured Data Extraction · Medium. An extraction pipeline reports 97% accuracy overall, and the team proposes removing human review. What should you check first?

A. Whether the model's stated confidence is above 90%

B. Whether the schema is enforced by tool use

C. Whether a larger model improves the overall figure

D. Accuracy by document type and field, using a stratified sample

A D (TS 5.5). An aggregate can hide weak segments, and those are where errors go unseen once review stops. A is uncalibrated, B removes syntax errors only, and C doesn't answer the question asked.

Q Q2 · Exam style, scenario: Customer Support · Medium. A customer writes, "I want to talk to a real person," though the agent could solve the issue easily. What should the agent do?

A. Solve it anyway, since that's quicker

B. Ask them to explain why they want a human

C. Hand off to a human, with a structured summary of the case

D. Run sentiment analysis, and escalate if the score is low

A C (TS 5.2). An explicit request for a human is a trigger, to honour directly. It's also the responsible AI principle in working form. A and B resist the request, and sentiment in D isn't the trigger.

Q Q3 · Attacks are 5 in 10,000 messages. A screen has 90% recall, and a 2% false-alarm rate. Of 1,000,000 messages, how many flags are there, and what share are real? What should a flag do?

A Attacks: 500, caught: 450. Innocent: 999,500, flagged: 19,990. Flags total 20,440, and about 2.2% are real. So a flag should cause closer inspection or stricter handling, not a block.

Q Q4 · Compare a jailbreak with an indirect prompt injection, and say which of them your architecture is more responsible for.

A A jailbreak gets the model to produce what it's trained to refuse, usually from a hostile user. Injection gets the model to follow someone else's instructions, usually from content it reads, and the harm is in actions and data. Provider training and classifiers carry much of the first, and your design (scoped tools, argument checks, no way out) carries much of the second.

Revision - Lesson 5

→ Jailbreak: forbidden output. Injection: someone else's instructions. Different main defenses.

→ Refusal is learned behaviour, so it can be argued with. No rule engine sits beneath it.

→ Published classes: encoding, role-play, substitution, injection, many-shot.

→ Constitutional Classifiers: 86% to 4.4% on synthetic attacks, refusals up 0.38%, compute up 23.7%. Beaten within a week in public.

→ Base rates: at rare attack rates, most flags are false, even at 95% recall.

→ A flag should mostly mean a closer look. A missing verdict must fail closed.

→ Checks against facts are exact, and cover only what they check.

→ Output can't be retracted once streamed, so buffer risky routes.

→ Screens of the same kind fail together. Measure overlap.

→ Responsible AI as work: disclosure, a human route, appeals, segments, an owner, and a kill switch.

Your notes

Module Revision

Self-assessment

#	Can you...
1	Explain why an agent can pass every classic control and still lose data?
2	Give the five threat-modelling questions?
3	List seven paths by which text reaches a model?
4	Explain why a Markdown image is a way out?
5	Rank sources by trust, including customer-written database fields?
6	State the lethal trifecta, and the second dangerous pair?
7	Separate direct attacks from indirect?
8	Explain why parameterized queries can't exist for prompts?
9	Read Anthropic's browser figures correctly, with their limits?
10	Explain why attackers benefit from retries?
11	Sort defenses into those that lower a rate and those that cap damage?
12	Explain why destination allowlists leave a refund tool exposed?
13	Give Anthropic's rules for placing untrusted content?
14	Explain why instructions don't belong in tool results?
15	Describe plan-first, dual-model and structured extraction, and their costs?
16	Give the four defects found in AI08's refund policy?
17	List the five dimensions of least privilege?
18	Show why a text check on paths fails?
19	Explain why allowlisted commands can still read private keys?
20	State Claude Code's rule evaluation order?
21	Explain why a Bash deny rule isn't a security boundary?

#	Can you...
22	Explain why a sandbox needs both filesystem and network limits?
23	Describe the credential proxy pattern?
24	Explain token passthrough and the MCP rule against it?
25	Map every store a prompt can reach?
26	Design a safe key for a semantic cache?
27	Explain what happens when an ineligible feature is used under ZDR?
28	Say why redaction can't be the only PII control?
29	Explain rights as an intersection?
30	Say what a hash chain proves, and what it needs?
31	List where data must be deleted?
32	Distinguish a jailbreak from an injection?
33	Quote Constitutional Classifiers' benefit and costs?
34	Do base-rate arithmetic for a guardrail?
35	Explain why a screen with no verdict must fail closed?
36	Turn three responsible AI principles into things you'd build?

Where this module is used

Idea	Where it returns
Red-team sets, gates, and evals	AI10
Rate limits, spend, kill switches, and incidents	AI10
Guardrail metrics and dashboards	AI10
Permissions, sandboxing, managed settings	AI12
Hooks, enforcement and scoped tools on the exam	AI13

Working space